MACHINE LEARNING AND AI FOR ANALYTICAL CHEMISTRY

Part 5: LLMs as adaptive research tools

M2 Sciences analytiques - 3 h tutorial

Roadmap for Part 5

1 Principles and good practice

What an LLM is, strengths and weaknesses, hallucination and verification, prompt hygiene, the fraud-policy boundary

2 Using LLMs for the work

Code generation, literature exploration, document automation, and the assessed project

Principles and good practice

Fluent, useful, and sometimes confidently wrong

Strengths and weaknesses

Good at

- drafting and rewriting text
- explaining concepts
- generating and debugging code
- translating between formats
- getting oriented quickly

Weak at

- inventing facts and references
- sensitivity to how it is asked
- being subtly wrong, but fluent
- knowledge cutoff and gaps

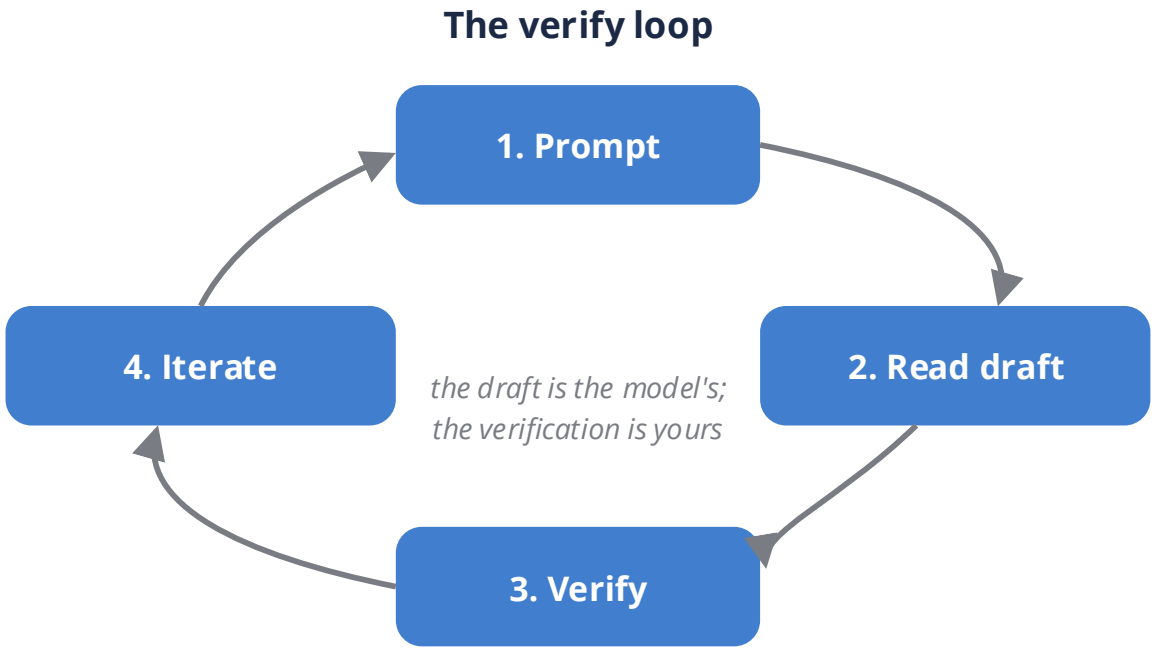
Example: asked for a method's reference, an LLM may return a perfect-looking citation with a DOI that does not exist.

Hallucination and verification

The model has no notion of truth; it produces what is likely.

So every output that matters is verified independently: run the code, check the reference exists, confirm the number against the data.

The draft is the model's; the verification is yours.



Example: an LLM writes a peptide-mass function; you test it on a peptide whose mass you know before trusting it.

Prompt hygiene

The quality of the answer depends on the prompt.

Give context, state the goal precisely, supply an example, specify the format and length, and ask for steps when useful. Iterating on the prompt is normal.

Anatomy of a good prompt

Context	what the data are and where they come from
Goal	the precise task, stated unambiguously
Example	a small input/output sample
Format	the shape of the answer (DataFrame, plot, length)
Reasoning	ask for steps when the task is non-trivial

Example: "read this CSV of m/z and intensity, find peaks above 5% of the base peak, return a DataFrame sorted by intensity" works; "analyse my data" does not.

The fraud-policy boundary

Required and assessed

In the project: use LLMs, and document them. An appendix of your key prompts and how you verified each output is part of the deliverable.

Prohibited

In the terminal written exam: no generative AI, no connected devices, per the programme regulations. Unauthorised use is a fraud offence.

The rule is deliberately asymmetric: documented use where it is the skill, none where it is the test.

Using LLMs for the work

Code, literature, documents, and the project

Code generation

The most reliable scientific use: generate code you then run and test.

Scripts for data processing, analysis and visualisation, sometimes whole small apps. Always: describe precisely, get code, run it, test on a known case, iterate.

Example: a cell that loads a chromatogram, baseline-corrects, detects peaks and plots them, checked against a manual count on one file first.

Literature and knowledge exploration

Useful to get oriented, summarise a concept, or suggest search terms.

Not a citation source: every reference must be confirmed in a real database before you use it.

Example: use an LLM to explain DDA vs DIA and suggest reviews, then find and read those reviews in PubMed rather than citing the model.

Improving and automating documents

LLMs help draft, restructure and tidy reports, method sections, captions and repetitive text.

The author stays responsible for all content, including every claim and number.

Example: draft a methods section from bullet points, then check every instrument setting and value against the actual experiment.

The assessed project

Introduced and started here.

Produce a documented Colab notebook (data processing, analysis, visualisation) plus a short report, with an appendix recording your key prompts and how you verified each output. Options, structure and the rubric are in the assessment folder.

Example: the LLM-use appendix is both the good-practice habit from this part and the integrity safeguard for the assessment.

Key takeaways

- An LLM predicts plausible text; it has no built-in notion of truth.
- Verify everything that matters: run code, confirm references, check numbers.
- Good prompts give context, goal, example, format; iterate.
- Strongest scientific use is code you then run and test.
- Required and documented in the project; prohibited in the exam.